DOMAIN 4 // ATTACKS & EXPLOITS

COMPTIA PENTEST+ PT0-003 · OBJECTIVES V3.0 · ATTACK-TYPE & TOOL REFERENCE

35% OF EXAM · ~32 QUESTIONS · 10 OBJECTIVES

THE TOOLS THAT SHOW UP EVERYWHERE — MEMORIZE THESE FIRST

TOOL	ROLE	APPEARS IN
Mimikatz	Credential dumping from LSASS, pass-the-hash, ticket extraction	4.4, 5.2
CrackMapExec (CME)	Validate / spray credentials across many hosts at once	4.2, 4.3
Impacket	Python toolkit for AD / SMB protocol attacks — the recurring "glue" tool	4.2, 4.10, 5.2
Responder	LLMNR / NBT-NS poisoning to capture NTLM hashes	4.2, 4.3
Scapy	Universal packet crafting across network, wireless and OT protocols	4.9, 4.10

WHY THIS MATTERS

If a question is ambiguous and one of these five plausibly fits, it is often the intended answer — they are the most canonical, most reused tools across CompTIA's scenarios.

4.1 PRIORITIZE & PREPARE ATTACKS — GIVEN A SCENARIO, ANALYZE OUTPUT

No tool list — this objective is analysis and planning. It decides *which* confirmed finding to attack and *how* to prepare. The metrics cluster is the highest-yield part.

DESCRIPTORS AND METRICS — KEEP THE FOUR STRAIGHT

METRIC	WHAT IT IS	KEYWORD / WHAT TO EXPECT
CVE	Identifier for a *specific* vulnerability instance	"which vulnerability" / a CVE-2024-xxxx string → the identity, not the score
CWE	Classifies the underlying *weakness type* (e.g. CWE-89 SQLi)	"category / class of flaw", "root weakness" → CWE
CVSS	Severity *score*, base 0–10	"how severe" → CVSS. Base = intrinsic; context-free
EPSS	Probability the vuln is *exploited in the wild*	"how likely to be exploited", "prioritize by real-world risk" → EPSS

THE PRIORITIZATION TELL

Severity alone does not set priority. **EPSS + public-exploit availability** is what elevates a moderate-CVSS finding above a theoretically critical one nobody is attacking. A CVSS 9.8 on an isolated box can rank below a 6.5 on the billing path — that is business impact (1.2) doing the work.

- **Target prioritization** — high-value assets, end-of-life software, default configs, running services, vulnerable encryption, defensive capabilities.
  Keyword: "which target first" → weigh value × exploitability × likely detection.
- **Capability selection** — tool selection, exploit selection and customization (incl. code analysis), documentation (attack path, low-level diagram, storyboard), dependencies, scope limitations, labeling sensitive systems.
  Keyword: "before running a public exploit" → review the code first (3.2). "Production + memory-corruption exploit" → authorize / schedule / safer path.

4.2 NETWORK ATTACKS — GIVEN A SCENARIO, PERFORM

ATTACK TYPE	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
Default credentials	CME, Hydra, Metasploit	"tried admin/admin", vendor default not changed
On-path attack	Wireshark/tcpdump, Responder	Intercepting traffic between two parties (the term "MITM" reworded)
Relay attack	Responder + Impacket (ntlmrelayx)	Captured auth *forwarded* to a different target — not cracked
Certificate services	Impacket, Certipy	AD CS / ESC abuse at the network layer
Misconfigured services	Nmap, Metasploit	Generic exploitation of a bad config
VLAN hopping	Nmap, packet crafting (Scapy)	Escaping segmentation — switch spoofing or double tagging
Multihomed hosts	Nmap, CME	Pivoting off a host with 2+ interfaces bridging segments
Share enumeration	CME, Impacket	SMB shares specifically
Packet crafting	Scapy, Nmap NSE	Building custom, precise packets

PATTERN TO REMEMBER

"Enumerate / authenticate broadly across a subnet" → **CME**. "Craft / inject custom packets" → **Scapy**. "Exploit a known service vulnerability" → **Metasploit**. "Relay a captured NTLM hash" → **Responder + Impacket**.

RELAY VS CRACK — THE DISTINCTION THEY TEST

A *relay* forwards captured authentication to another host in real time and never learns the password. *Cracking* (4.3) takes the captured hash offline to recover the plaintext. SMB signing defeats the relay; it does nothing against cracking.

4.3 AUTHENTICATION ATTACKS — GIVEN A SCENARIO, PERFORM

ATTACK TYPE	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
MFA fatigue	CME, custom scripts	Spamming push notifications until the user approves
Pass-the-hash / ticket / token	Mimikatz, CME, Rubeus	Reusing a stolen credential *artifact* directly — no plaintext
Kerberos attacks	Rubeus, Mimikatz	Kerberoasting, AS-REP roasting, golden/silver tickets
LDAP injection	Burp Suite	Manipulating an LDAP query through app input
Dictionary / brute-force / mask	hashcat, John the Ripper	**Offline** — you already hold the hash file
Password spraying	CME, Hydra	Many usernames, *one* password — stays under lockout
Credential stuffing	Hydra, CME, Medusa	Reusing breached user:pass pairs
OIDC / SAML attacks	Burp Suite	Manipulating federated tokens / assertions

PATTERN TO REMEMBER

hashcat / John = offline cracking (you have the hash). **Hydra / Medusa** = online / live login brute-forcing against a service. **CME** = validate credentials across many hosts. **BloodHound** = maps AD attack paths, does not attack directly.

SPRAY VS STUFF VS BRUTE

Spray = one password against many accounts (evades lockout). *Stuff* = known breached pairs against many services (relies on reuse). *Brute / dictionary* = many guesses against one account (trips lockout online, so usually done offline).

4.4 HOST-BASED ATTACKS — GIVEN A SCENARIO, PERFORM

ATTACK TYPE	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
Credential dumping	Mimikatz	Pulling creds from LSASS memory / SAM
Privilege escalation	Seatbelt, PowerSploit, WinPEAS	Finding local escalation paths
Kerberos ticket abuse	Rubeus	Requesting / manipulating tickets on the host
Certificate abuse (AD CS)	Certify	Abusing AD Certificate Services
Unquoted service path	PowerUp, manual	Space in an unquoted path + writable dir → escalation
Process hollowing / injection	Custom, C2 frameworks	Running code inside a legitimate process
Kiosk / shell escape	Native OS features	Breaking out of a restricted shell or kiosk UI
Remote execution	PsExec, Evil-WinRM	Executing commands on a remote host *once you have creds*
Evasion / execution	LOLBins (rundll32, mmc, msbuild)	Native signed binaries to bypass allow-lists

PATTERN TO REMEMBER

Mimikatz = credential dumping / pass-the-hash. **Rubeus** = Kerberos ticket manipulation. **Certify** = AD CS abuse. **PsExec / Evil-WinRM** = remote command execution once you have creds. **LOLBins** = allow-list bypass via trusted native binaries.

4.5 WEB APPLICATION ATTACKS — GIVEN A SCENARIO, PERFORM

ATTACK TYPE	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
SQL injection	sqlmap	Automated SQLi detection / exploitation — **always sqlmap**
Command injection	Burp Suite, manual	App passes input to a shell
XSS / server-side template injection	Burp Suite / ZAP	Reflected/stored/DOM; template engine evaluates input
Directory traversal / RFI / LFI	Gobuster/DirBuster, Burp	Find hidden dirs first, then include/traverse to a file/web shell
SSRF / CSRF	Burp Suite / ZAP	SSRF = server fetches attacker URL (metadata!); CSRF = forged state-changing request
IDOR / session hijacking	Burp Suite / ZAP	Changing an object ID / stealing a session
Deserialization	Burp, ysoserial	Untrusted serialized object → code execution
JWT manipulation	Burp Suite	`alg:none`, weak signing key
API abuse	Postman	Scenario explicitly names an API
WordPress-specific	WPScan	Any mention of WordPress
Exposed secrets in code	TruffleHog	Keys / creds in repos — not a live app

> PATTERN TO REMEMBER
>
> **sqlmap** = SQLi, always. **WPScan** = WordPress. **Gobuster/DirBuster** = finding hidden content (a first step, not the exploit). **Burp Suite / ZAP** = the generalist for everything else (CSRF, IDOR, session hijacking, JWT, XSS, SSRF). **Postman** = API-specific. **TruffleHog** = secrets in code.

4.6 CLOUD-BASED ATTACKS — GIVEN A SCENARIO, PERFORM

ATTACK TYPE	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
Metadata service attack	curl, Pacu	SSRF to `169.254.169.254` → steal instance credentials
IAM misconfiguration	Pacu, Prowler, ScoutSuite	Over-permissive policy, privilege escalation via IAM
Exposed storage buckets	Prowler, ScoutSuite, vendor CLI	Public S3 / blob storage
AWS exploitation	Pacu	Post-exploitation framework, AWS-only
Container escape	manual, Docker Bench (audit)	Breaking out of a container to the host
Container / K8s hardening audit	Docker Bench, Kube-hunter	Docker Bench = Docker config vs best practice; Kube-hunter = K8s attack surface
Supply chain / image tampering	manual, cloud-native tools	Poisoned base image or dependency
Config auditing (multi-cloud)	Prowler, ScoutSuite	Broad misconfiguration scanning — assessment, not exploitation

> PATTERN TO REMEMBER
>
> **Pacu** = AWS exploitation (a fish in AWS's ocean). **Prowler / ScoutSuite** = cloud posture / config auditing (assessment). **Kube-hunter** = Kubernetes. **Docker Bench** = Docker hardening checks. Metadata-service SSRF is the single most-tested cloud attack — know the `169.254.169.254` address.

4.7 WIRELESS ATTACKS — GIVEN A SCENARIO, PERFORM

ATTACK TYPE	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
Wardriving / SSID discovery	WiGLE.net, InSSIDer, Kismet	Mapping / discovering nearby networks (recon)
Evil twin / captive portal	WiFi-Pumpkin	Rogue AP impersonating a legitimate SSID
Deauthentication	Aircrack-ng (aireplay-ng)	Forcing a client off to capture the reassociation handshake
Handshake capture / cracking	Aircrack-ng	The core WPA/WPA2 cracking suite — the classic answer
WPS PIN attack	Reaver, Bully	Brute-forcing the 8-digit WPS PIN
Rogue proxy auto-discovery	WPAD + Responder	Exploiting proxy auto-config trust

PATTERN TO REMEMBER

Aircrack-ng = capturing / cracking WPA handshakes and deauth (the classic answer, and the only one that transmits). **WiFi-Pumpkin** = rogue AP / evil twin. **Kismet** = passive continuous scanning / wardriving. **WiGLE.net** = crowdsourced location database (recon, not attack).

4.8 SOCIAL ENGINEERING ATTACKS — GIVEN A SCENARIO, PERFORM

ATTACK TYPE	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
Phishing campaigns	Gophish, SET	Sending / tracking phishing email at scale
Credential harvesting (MFA bypass)	Evilginx	Reverse-proxy phishing capturing *live session tokens* — defeats MFA
Browser-based exploitation	BeEF	Hooking a victim's browser once they visit a page
Vishing / smishing / whaling	SET, manual	Voice / SMS / exec-targeted variants of phishing
OSINT for targeting	theHarvester, Maltego, Recon-ng	Building target lists (emails, names)

PATTERN TO REMEMBER

Gophish = phishing campaign management (send, track opens/clicks). **Evilginx** = advanced phishing that steals live session tokens, *defeating MFA* — the key differentiator from Gophish. **BeEF** = browser hooking. **SET** = older all-purpose social-engineering toolkit. Remediation for the whole family is **user training** (operational, 1.5).

4.9 SPECIALIZED SYSTEMS ATTACKS EXPLAIN

ATTACK TYPE	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
Mobile app analysis	MobSF	Static / dynamic mobile app scanning
Mobile runtime manipulation	Frida	Instrumenting / hooking a *running* app — more interactive than MobSF
Android-specific testing	Drozer, ADB	Android app / device interaction
AI attacks	manual	**Prompt injection** and **model manipulation** — new to PT0-003
OT / embedded (CAN bus, Modbus, register manipulation)	Scapy, tcprelay	Crafting custom protocol packets; replay / plaintext attacks
NFC / RFID / Bluejacking / BT spamming	Bluestrike, Scapy	Short-range wireless against a device
Traffic capture (any protocol)	Wireshark/tcpdump	Universal packet analysis

> **PATTERN TO REMEMBER**
>
> **MobSF** = mobile app static/dynamic scanning. **Frida** = runtime instrumentation / hooking (more advanced than MobSF). **Drozer** = Android attack-surface testing. **Scapy** = the universal answer for CAN bus / Modbus / any custom protocol crafting. The bluetooth tool on the objectives is **Bluestrike**.

4.10 SCRIPTING TO AUTOMATE ATTACKS GIVEN A SCENARIO

CATEGORY	TOOL(S)	PATTERN / KEYWORD TO LOOK FOR
PowerShell AD attacks	PowerSploit, PowerView, PowerUpSQL, AD search	AD enumeration / exploitation via PowerShell
Python attacks	Impacket, Scapy	Protocol interaction, packet crafting
Bash automation	Input/output management, data manipulation	Glue and parsing on Unix hosts
Breach & attack simulation (BAS)	Caldera, Infection Monkey, Atomic Red Team	Simulating real adversary TTPs, mapped to MITRE ATT&CK

> **PATTERN TO REMEMBER**
>
> **Impacket** = Python AD/SMB protocol toolset, the recurring glue tool. **PowerView** = AD enumeration via PowerShell. **Caldera / Atomic Red Team / Infection Monkey** = automated adversary emulation — "simulate real attacker behavior automatically, mapped to known TTPs" → BAS, not manual pentest tools.

> **CORRECTION FROM THE SOURCE DRAFT**
>
> The PT0-003 objectives list **PowerSploit, PowerView, PowerUpSQL, and AD search** for PowerShell — "Empire" is not on the list. Under 4.9 the Bluetooth tool is **Bluestrike**, not "Bluecrack" (which is not a real tool).

35% of the exam — the one to over-prepare. // Match the attack type to the artifact, then the tool.